



Transcript

Week 22: Generative AI- Part 1

Video 1 - Introduction to Generative AI

Have you ever wondered how an A-I system can write a poem, generate realistic images, or even compose music? Generative A-I is not just about analysing data—it's about creating something new from it. You'll uncover how these systems work and why they're shaping the future of creativity and innovation.

Let's begin with a fundamental question—what exactly is Generative AI, and what makes it so unique?

Generative AI refers to a class of artificial intelligence models that don't just analyse or classify data. Instead, they're designed to actually create new content. Think of it as a creative powerhouse. These models can produce original text, images, audio, and even computer code—by learning from patterns in existing data.

For example, ChatGPT can generate essays or poems just from a few typed prompts. Tools like Midjourney or DALL-E can turn text descriptions into highly realistic images. And GitHub Copilot can write functional code snippets based simply on your intent—no manual programming needed.

Now, what makes these models powerful isn't just that they copy what they've seen. Instead, they learn the underlying patterns and relationships from large amounts of training data. That allows them to generate outputs that often feel surprising, creative, or even original.

In contrast, traditional AI systems are designed for narrow, specific tasks—like detecting spam emails or predicting house prices. These systems are known as discriminative models. They work by recognising and labelling what already exists. But they can't invent or imagine something new.

Generative AI takes a different approach. It moves from imitation to innovation. While it draws inspiration from what it has seen, it also goes beyond—producing content that didn't exist before.

So, in essence, Generative AI changes the game. It's not just about recognising patterns. It's about using those patterns to create new ideas—helping machines surprise us, inspire us, and even reshape how we think about creativity.

To understand where we are today with Generative AI, it helps to look back at how this field evolved. The journey spans more than seventy years, starting in the 1950s.

In the first phase—from the 1950s to the 1980s—AI models were quite simple. One major example was the Markov chain. It's a model that uses probabilities to predict the next item in a sequence based on the current one. These early models couldn't generate content but laid the foundation for how AI could learn from data.

Then came the second phase in the 1980s through the early 2000s. This is when neural networks started gaining attention. These models mimicked the human brain by using connected layers of artificial neurons. But at that time, they were still shallow—limited in complexity and scale. They could spot patterns in data but couldn't generate anything rich, like realistic images or fluent language.

The real breakthrough came in the 2000s and continues today with deep learning. This



new generation of AI uses multi-layered neural networks trained on huge datasets. They can now learn much more complex patterns and generate realistic, creative outputs—like fluent text, lifelike images, human-sounding voices, and even music.

To make this evolution more relatable: think about a chatbot from the 1990s that gave short, pre-written answers. Now compare that to ChatGPT, which can write essays, solve problems, or hold a conversation in real time. Or picture early computer graphics versus tools like Midjourney, which can now generate photo-realistic art from a text description.

So, in summary, Generative AI has evolved from simple rule-based systems to deep learning-powered models that can create content from scratch. And it's this third stage—deep learning—that's unlocking the most advanced, creative capabilities we see today.

Let's take a closer look at what makes a generative AI system work. There are five key components that power these systems and help them create original content.

First, we have data and datasets. These are the building blocks of any generative model. To learn effectively, the model needs large and diverse datasets—made up of text, images, audio, code, or any other kind of input. The more variety and volume, the better the model can understand patterns and produce high-quality, realistic results.

Next is the model architecture. This refers to the structure of the AI model itself. Generative AI uses specialised architectures like GANs, which stands for *Generative Adversarial Networks*, V-A-Es, or *Variational Autoencoders*, and transformer models like G-P-T, or *Generative Pretrained Transformer*. Each one has a different way of learning from data, but all are designed to create new content rather than just recognise existing patterns.

The training process is where learning happens. Models are trained using either supervised or unsupervised learning. In supervised learning, the model is shown both input and correct output—like an image and its label. In unsupervised learning, the model finds patterns without being told what to look for. This step teaches the model how to generalise from examples and create something new.

Then we have loss functions and optimisation. These are techniques that help the model improve as it trains. For images, we often use F-I-D, or *Fréchet Inception Distance*, to measure how realistic the output is. For text, we look at likelihood scores to see how well the model predicts the next word or phrase. These tools help fine-tune the model's performance.

Finally, we come to ethics and bias. This is a crucial part of generative AI. If the training data includes biased or harmful content, the model can reproduce that. So, developers must actively look for and reduce bias, and ensure that the AI is used responsibly. This helps build systems that are not only powerful, but also fair and safe.

So in summary, these five components—data, architecture, training, optimisation, and ethics—work together to build creative AI systems that generate useful, convincing, and responsible content.

Let's now explore the core ideas that power how generative AI learns, creates, and evolves.

At its heart, generative AI is all about creating new data—synthetic text, images, audio, or even video—that closely mimics real-world content. This could mean writing a poem, generating a photo-realistic face, or producing music from a prompt.

But how does it do that?



The process begins with learning from data. Models are trained on large and varied datasets. They scan for patterns and features—things like sentence structure, colour gradients, or rhythms in music. This helps them understand the structure of real-world content so they can create something similar.

A key concept behind this is called latent space representation. Think of it like compressing information. The model takes what it has learned and reduces it into a simplified form—a kind of map of abstract ideas. From this compressed space, it can decode and generate entirely new content that still relates to what it originally learned from.

Then comes optimisation and loss functions. These guide the model during training. Metrics like likelihood—which measures how well a text prediction fits—and F-I-D, or *Fréchet Inception Distance*, used for images, help the model refine its outputs. The goal is to reduce the gap between generated results and high-quality examples.

Once trained, the model focuses on creating creative and context-aware outputs. That means it doesn't just generate random content. Instead, it responds to inputs in a meaningful way. For instance, if you give it a prompt like “a sunset over a mountain,” it generates results that match that idea—based on everything it has learned.

Lastly, the model uses exploration and sampling to improve creativity and variety. It explores the latent space and samples different combinations to create unique, diverse outputs each time. That's why even with the same prompt, generative AI can surprise us with slightly different results.

Together, these core principles form the engine of generative AI—helping machines learn from data, compress knowledge, optimise performance, and ultimately create content that feels creative, useful, and new.

Can you take these core principles and turn them into something creative? Start thinking like a machine—and designing like a creator.

Video 2 - Generative AI Model Types

Which model would you choose to generate an artwork, a voice clip, or a piece of writing? Generative A-I uses different model types, each designed for specific kinds of creativity. Knowing how they work helps you build smarter, more effective AI solutions.

Let's take a closer look at how Generative Adversarial Networks, or GANs, work. They're one of the most powerful tools in generative AI—and what makes them special is that they involve two neural networks working in competition with each other.

The first network is called the generator. Its job is to create synthetic data that looks as real as possible—whether it's an image, a piece of music, or a block of text. The second network is the discriminator. Its task is to tell whether the data it sees is real, from the training set, or fake, created by the generator.

These two networks are locked in what's called adversarial training. The generator tries to create data that's good enough to fool the discriminator. At the same time, the discriminator keeps getting better at spotting what's fake. It's like a game where one player tries to create convincing forgeries while the other sharpens their detection skills.

As this back-and-forth continues, both networks improve. The generator learns to produce data that's increasingly realistic, and the discriminator becomes more accurate. The result is a system that can generate very high-quality, novel content—whether it's a photo-realistic image of a person who doesn't exist or a painting in the style of Van Gogh.



GANs are used in many real-world applications. They can generate lifelike faces, transform images from one style to another, like turning a sketch into a full-colour image, or augment datasets to improve the training of other machine learning models.

This dynamic feedback loop between the generator and discriminator is what gives GANs their creative power. It's competition with a purpose—and that's what makes them such an effective way to produce new, convincing data.

Let's now explore another powerful tool in generative AI—Variational Autoencoders, or V-A-Es.

Unlike traditional autoencoders, which focus mainly on compressing and reconstructing data, V-A-Es take a probabilistic approach. That means they don't just memorise and reproduce—they learn to represent the input data in a compressed, lower-dimensional format called latent space.

This latent representation captures the most important patterns and features of the original data. For example, if you feed in handwritten numbers, the model doesn't memorise every pixel. Instead, it learns key characteristics like stroke thickness or digit shape—and stores this information in a much smaller form.

Once trained, the V-A-E can sample from this latent space to generate new outputs that are slightly varied but still resemble the original data. This is where the probabilistic generation comes in. The model uses probability to introduce slight variations in the output, helping it produce new examples that look realistic but aren't exact copies.

Because of this creative flexibility, V-A-Es are used in several applications. They're common in image generation, where they create new visuals based on patterns in the training set. They're also used for text-to-image synthesis, where a description like "a cat wearing sunglasses" is turned into a visual. And they help with anomaly detection—for example, flagging unusual data points that don't match the normal pattern, which is useful in areas like fraud detection or medical diagnostics.

So in short, V-A-Es blend compression with creativity. They learn a simplified version of the data, use probability to reimagine it, and generate useful, diverse outputs in a wide range of settings.

Let's now explore one of the biggest breakthroughs in natural language processing—Transformer language models.

Transformers are a special kind of neural network architecture built specifically to understand and process human language. What makes them so powerful is a feature called attention mechanism. This allows the model to focus on different parts of a sentence—picking up on which words relate to each other, even if they're far apart in the text.

For example, in the sentence "The cat, which had been hiding under the couch, finally came out," the model learns that "cat" is connected to "came out," even though many words are in between. This ability to capture long-range relationships is something older models like R-N-Ns or L-S-T-Ms struggled with.

Thanks to this attention mechanism, transformers can understand the deeper context of words, not just their position. This has led to major leaps in how machines handle tasks like machine translation—for instance, converting English text into French, or text summarisation, where long documents are condensed into short, meaningful summaries. They also excel at question answering, where the model reads a passage and generates a relevant response.

Transformers have truly revolutionised NLP, or *natural language processing*. Models like G-



P-T and B-E-R-T are both built on this transformer foundation—and today, they’re used in a wide range of applications, from virtual assistants to search engines and beyond.

In short, transformers changed the game because they understand words in context, not just in order. That’s what allows them to produce smarter, more accurate, and more natural responses.

Let’s explore where Generative AI is making a real impact in the world today.

One major area is image generation. Here, AI models take text prompts—like “a futuristic city at sunset”—and turn them into highly detailed, realistic, or artistic visuals. These systems can also enhance or completely transform existing images. This has huge value in media, content creation, design, and visual storytelling.

Another powerful application is text generation. Generative AI can write stories, generate code, summarise documents, translate between languages, or even simulate dialogue. It’s being used in tools like chatbots, writing assistants, and coding platforms to support creativity and communication.

Music and audio generation is also gaining momentum. AI models can compose original songs, generate sound effects, and create voiceovers. For example, an AI could create background scores for a video game or read a script aloud in a human-like voice. This opens new possibilities for entertainment and content production.

Beyond creative fields, Generative AI is transforming science and industry. It’s being used in drug discovery by generating molecular structures, in personalised learning by creating adaptive educational content, and in product design through 3D modelling or material simulation.

In short, generative AI is not limited to just one domain. From art to science, from writing to research, it’s shaping the future by helping us create, design, and innovate across disciplines.

Let’s take a look at how Generative Adversarial Networks, or GANs, are used to create realistic images.

A GAN works by combining two neural networks—a generator and a discriminator. The generator’s job is to create images from scratch—images that look real, even though they’re completely made up. The discriminator’s job is to evaluate those images and decide whether they’re real or fake—whether they come from the training data or were created by the generator.

These two networks engage in a process called adversarial training. The generator keeps trying to produce better and better fake images—ones that can fool the discriminator. At the same time, the discriminator keeps improving its ability to detect which images are fake. It’s a kind of feedback loop where both networks push each other to improve.

Over time, this competition leads to realistic output. The generator becomes so good at creating images that even the discriminator can’t always tell them apart from the real thing.

This approach is incredibly effective and is used in many creative and technical areas. For example, GANs can help with art creation, photo editing, and even deepfake generation—where a person’s face or voice is digitally recreated with high accuracy.

So, in summary, GANs create realistic images by training two networks to compete and improve. The result? High-quality visuals that are almost indistinguishable from real ones.



Let's look at how language models generate text that feels natural, fluent, and meaningful—just like a human would write.

Language models are powerful tools trained to produce high-quality text. They can write anything from poems and short stories to technical summaries, answers to questions, or even translations between languages.

The most advanced models today are called L-L-MS, or Large Language Models. These models have been trained on massive amounts of text data—books, articles, websites, transcripts, and more. This extensive training helps them understand how language works—how grammar, sentence flow, and even tone or style change depending on the context.

Thanks to this, L-L-MS can produce coherent, context-aware outputs. For example, they can complete a paragraph without losing the thread, summarise long documents into clear bullet points, or translate a sentence while preserving its original meaning and nuance.

So how do they do it? These models learn patterns in language. They don't just memorise—they predict what word should come next, based on everything they've seen. That prediction is what helps the model sound so fluent and relevant.

Because of their versatility, language models are used in a wide range of real-world applications—from chatbots and customer support to content writing, coding assistance, and language learning platforms.

In short, L-L-MS learn to write like us—because they've read so much of what we've written. That's what makes them such powerful tools for communication and creativity.

Generative AI is transforming how we create and experience music, sound, and speech.

Let's start with music composition. AI models today can generate original music from scratch. This could be a simple melody or a complex orchestral piece. Imagine an AI that creates custom background scores for films, video games, or even music based on your mood. That's now possible thanks to generative models.

Next, we have sound effects. AI can generate realistic and unique audio effects for games, movies, and multimedia. Whether it's footsteps in a video game, rain falling in a film, or futuristic ambient noise in virtual reality, AI can produce high-quality sounds that enhance immersion and storytelling.

Another major breakthrough is in speech synthesis. AI models can now generate human-like speech that sounds natural and expressive. This is the technology behind virtual assistants, audiobooks, and personalised voiceovers. It makes machine-generated speech feel more real and engaging.

So in summary, from music and sound to speech that feels human, Generative AI is changing the way we hear. It brings new creativity to entertainment, media, and communication—pushing boundaries in ways that were not possible before.

Generative AI is full of promise—but it also comes with real challenges that we need to understand to use it responsibly.

The first challenge is data dependence. Generative models learn from huge amounts of training data. If that data is limited, poor in quality, or biased, the outputs will reflect those flaws. For example, if a model is trained mostly on formal text, it might struggle to generate casual conversation—or worse, it may reflect harmful stereotypes from the training content.



Next is bias and fairness. Since these models learn from real-world data, they can pick up patterns that include unfair assumptions or underrepresented voices. When that happens, the model can unintentionally amplify bias—especially in areas like job recruitment, education tools, or financial services.

A third issue is lack of explainability. Even when the model works well, we often don't know why it made a certain choice. For instance, if it generates a wrong or inappropriate sentence, it's difficult to trace the source of that mistake. This lack of transparency can make it hard to correct errors or improve reliability.

Finally, we come to control and predictability. Generative AI is great at being creative—but that creativity can be hard to manage. If you want a response in a formal tone or need an answer in a specific format, the model might not follow the instructions exactly. That makes it tough to use in situations that demand precision and consistency.

Understanding these limitations is key—not to hold AI back, but to use it wisely and ethically as we continue building smarter, more reliable systems.

As we develop and use generative AI, it's critical to think about ethics—how we ensure that this technology is used responsibly and fairly.

One of the biggest concerns is bias and fairness. Because generative AI learns from real-world data, it can also learn real-world biases. If that data contains stereotypes or imbalances, the model may unintentionally reflect or amplify them. This could lead to unfair outcomes, especially in sensitive areas like education, healthcare, or hiring. So, it's essential to detect, reduce, and prevent bias to make sure the outputs are as fair as possible.

Another serious issue is misinformation and deepfakes. Generative AI can create extremely realistic fake content—images, videos, or voice recordings that appear authentic. While this technology can be used creatively, it also has the potential to mislead and deceive. That's why we need strong safeguards and detection tools to stop the spread of harmful or false content.

Privacy and data security are also key concerns. Generative models are often trained on large datasets, which can include sensitive personal information. Protecting this data—making sure it's stored safely and used with consent—is crucial to maintaining trust and respecting user rights.

Finally, we have to talk about job displacement. As generative AI gets better at tasks like content creation, summarising reports, or even generating code, it may start replacing some human roles. That doesn't mean jobs will disappear—but it does mean they'll change. Preparing for that change through education, reskilling, and policy support is a vital part of using AI ethically.

So, as we push the boundaries of what generative AI can do, we must also push ourselves to use it with care—balancing innovation with responsibility.

Let's look at some exciting trends shaping the future of generative AI.

First, we have Multimodal AI. These models go beyond just one kind of data. They can combine text, images, audio, and even video to create richer, more immersive content. For example, you could give the model a sentence like “a robot playing piano on a beach at sunset,” and it might generate an image or even a video based on that. Or it could turn an audio clip into an animated visual. This opens up new possibilities for creative storytelling, media production, and learning experiences.

Next is Interactive Generative AI. These systems are designed to work with you, not just



for you. Instead of passively receiving a result, users can collaborate with the AI—giving feedback, refining prompts, and co-creating content. That makes AI a creative partner rather than just a tool, helping enhance human ideas instead of replacing them.

The third trend is Personalised Generative AI. These models are being trained on individual user data, such as writing style, preferences, or browsing habits. The goal is to create tailored outputs—like custom music playlists, personalised summaries, or learning content that adapts to your needs. This makes AI feel more relevant, useful, and human-centred.

Together, these trends show that generative AI is evolving fast—becoming more versatile, more collaborative, and more user-focused. And that’s opening up incredible new possibilities across industries, from entertainment and education to design and healthcare.

Let’s look ahead at where generative AI is going—especially when it comes to the next generation of AI assistants.

First, we’ll see the rise of Advanced AI Assistants. These assistants will do more than just answer questions—they’ll handle complex tasks like writing personalised emails, creating custom playlists or designs, drafting proposals, or even composing music that matches your mood. They’ll understand context, learn your preferences over time, and generate content that feels uniquely yours.

Second, we’re moving toward more Personalised Experiences. Instead of one-size-fits-all responses, future AI assistants will generate content and recommendations tailored to you. For example, they could suggest outfits based on your past purchases, plan your meals based on your diet, or design a custom fitness routine. Everything becomes adaptive, based on your behaviour, style, or needs.

Third, new Forms of Interaction will change how we communicate with machines. You won’t need to speak in keywords or fixed commands. You’ll be able to have natural, flowing conversations with AI—just like you would with another person. Say something like “I need help creating a presentation about climate change,” and the AI will understand, ask clarifying questions, and start building content with you.

But with all this advancement comes a need for Ethical Considerations. As AI becomes more powerful and more personal, we must make sure it’s used responsibly. That means designing systems that protect privacy, reduce bias, and prevent misuse. Ethics can’t be an afterthought—they must guide how we build and use these technologies.

So, the future of generative AI is smarter, more personal, more human-like—and it must also be more responsible.

Video 3 - Variational Autoencoders (VAEs)

How can a machine create something entirely new, not by copying, but by reimagining what it has learned? Variational Autoencoders—or V-A-Es—do exactly that. By compressing data into a simpler form and then generating new variations, they open up powerful possibilities in AI creativity and detection.

Let’s now explore Variational Autoencoders, or V-A-Es—a powerful kind of generative model that combines two key ideas: autoencoding and probabilistic modelling.

First, let’s understand the autoencoder part. An autoencoder is a type of neural network that compresses high-dimensional input—like a large image—into a smaller, simpler representation called latent space. Think of it like shrinking a high-resolution image into a compact code. Then, the model learns to reconstruct the original image from that code,



trying to keep it as close to the original as possible.

Now, here's where VAEs take it further. Instead of encoding input into just one fixed point in that latent space, VAEs add a probabilistic layer. They don't learn a single point—they learn a whole distribution of where the encoded value could lie. Usually, this is modelled using a Gaussian distribution, also known as a bell curve. This lets the model sample different points from that distribution to generate new, slightly different outputs—ones that feel realistic, but aren't identical to the original data.

This flexibility makes VAEs powerful for generating new content. For example, in image generation, a VAE might learn how to represent dogs in latent space. Then, by sampling from that space, it can create new dog images that share common traits—but each one looks a little different.

VAEs are used in many real-world applications. In image synthesis, they help generate brand-new images based on patterns learned from training data. In text-to-image generation, you can feed in a sentence like “a cat riding a skateboard,” and the VAE helps generate a visual that matches. They also work in anomaly detection, where anything that strays too far from the learned distribution is flagged as unusual—useful in medical scans or fraud detection.

So, in summary, V-A-Es give us a flexible and creative way to generate new data. They take the idea of compressing information and add probability, allowing us to generate outputs that are realistic, diverse, and meaningful.

Let's start with autoencoders, one of the key building blocks behind many generative models.

Autoencoders are neural networks that learn to take input data — like an image, sound, or sentence — and compress it into a smaller, meaningful form. Then, they learn to reconstruct the original input from that compressed version.

This process has two parts.

First, the encoder. It transforms the input into a lower-dimensional format — we call this the latent code or latent representation. Think of it like summarising a detailed photo into just the key features that matter most.

Then comes the decoder. It tries to rebuild the original input using only that compressed summary. The better the encoder and decoder work together, the closer the output is to the original.

This compression–reconstruction process helps the model learn what features are truly important in the data.

Autoencoders are useful in several ways.

In dimensionality reduction, they help simplify complex data for analysis or visualisation.

In data compression, they shrink file sizes while preserving key information.

And for anomaly detection, they learn what “normal” data looks like — so if the input is unusual, the reconstruction fails, signalling something's off.

Here's an example.

Let's say we train an autoencoder on handwritten digits. It learns to compress and recreate digits like 3, 7, or 9. If we feed it a distorted or weird-looking digit, it won't reconstruct it well — and that mismatch tells us the input might be an anomaly.

In summary, autoencoders help us learn compact, meaningful versions of data. That makes them incredibly valuable in many AI tasks, especially when working with large or complex datasets.



Let's explore what makes Variational Autoencoders, or V-A-Es, unique—their probabilistic nature.

Unlike traditional autoencoders that map data to a single fixed point, V-A-Es map it to a distribution. This means they don't just compress information—they model the uncertainty in the data too.

V-A-Es use a process called variational inference. This helps them estimate what are called latent variables. These are hidden factors—like pose or lighting in an image—that explain why the data varies. We can't directly observe them, but the model learns them from the patterns in the input.

What's innovative is the probabilistic approach. Instead of turning each input into a fixed code, the encoder outputs a whole range of possibilities—a probability distribution. Most often, this is modelled as a Gaussian distribution, which you might know as a bell curve.

Now here's the magic: When we want to generate something new, we don't just take one point—we sample from this range. This lets V-A-Es produce diverse and realistic results.

Let's take an example. Imagine a V-A-E trained on human faces. Instead of memorising one version of a face, it learns a distribution over features—like smiling, looking left, or lighting angle. When we sample from this, the V-A-E can create new, varied versions of the same face.

So in summary, this probabilistic method lets V-A-Es model uncertainty, generate richer outputs, and better capture the complex patterns in real-world data.

Let's take a closer look at how a Variational Autoencoder, or V-A-E, is built. It combines a regular autoencoder structure with probabilistic modelling to generate new data.

The first part is the encoder. It takes input data, like an image, and compresses it into a simpler form. But unlike normal autoencoders, it doesn't just produce a fixed number—it gives us a distribution. That means it captures a range of possible values, showing uncertainty in the data.

Next is the latent space, also called the bottleneck. This is where the compressed information lives. Instead of a single number, we have a probability distribution, which gives us flexibility when generating new data.

To work with this randomness, V-A-Es use a clever technique called the reparameterization trick. It lets us sample from this distribution in a way that still allows the model to learn through backpropagation.

Then comes the decoder. It takes a sample from the latent space and tries to reconstruct the original data—or generate something new that looks realistic.

Finally, we use something called the E-L-B-O loss—that's short for Evidence Lower Bound. This helps balance two goals: how well the output matches the input, and how close the distribution is to a standard normal. This balance helps the model learn better and generate smoother results.

So, to sum up: V-A-Es are more than just compression tools. They learn useful patterns and generate new data with flexibility and variety—something traditional autoencoders can't do as well.

Let's break down how the encoder works inside a Variational Autoencoder, or V-A-E.

The encoder's job is to take input data—like an image or a sentence—and map it into a



probabilistic latent space. That's different from a regular autoencoder, which maps it to just one fixed number. Instead, a V-A-E uses probability to capture uncertainty and variation in the data.

Here's how it works.

The encoder doesn't give a single output. It gives us two:

First, the mean, written as Greek letter mu, μ . This tells us the centre of the distribution—the most likely value.

Second, the log variance, written as log sigma squared. This tells us how much the values can vary. Think of it as how wide or narrow the distribution is.

These two values define a probability distribution, usually a Gaussian or normal distribution. And we sample from this distribution to get a latent representation of the input.

To calculate these values, the encoder uses a fully connected neural network. That just means it has layers that take the input and learn the best values for mean and variance.

Why is this important? Because it lets the model represent uncertainty and create more varied, meaningful outputs. That's what makes a V-A-E so powerful compared to traditional autoencoders.

Now let's talk about the latent space, also called the bottleneck, in a Variational Autoencoder.

Unlike traditional autoencoders, a VAE maps input data to a probabilistic distribution instead of a fixed point. This means each point in latent space represents not just one encoding, but a range of possible values — capturing uncertainty in the data.

The encoder outputs two parameters:

Mean, written as the Greek letter mu, which gives the centre of the distribution, and log variance, written as log of sigma squared, which controls the spread.

From this distribution, we sample a latent variable, usually denoted as z .

This sampling step adds randomness, helping the model generate more varied and realistic outputs.

To make training work with this randomness, VAEs use a technique called the reparameterization trick. It lets the model pass gradients through the sampling process, which is essential for learning during backpropagation.

In short, this bottleneck lets the VAE explore different parts of the distribution and generate diverse outputs that still resemble the original data.

Let's break down the reparameterisation trick, which plays a key role in making sampling differentiable for learning.

The formula we use is: z equals μ plus σ times ϵ .

Now let's understand what each part means. μ is the mean of the distribution. It tells us the central value around which our data is expected to vary. σ is the standard deviation, which comes from the log variance. It reflects how spread out or uncertain the distribution is. ϵ is random noise, sampled from a normal distribution with a mean of zero and a standard deviation of one.

This combination lets us sample from the distribution, but in a way that remains differentiable. That means we can pass gradients through this sampling step and continue



training the model using backpropagation.

It allows gradient-based learning to work smoothly and supports full end-to-end training in variational autoencoders.

In short, the reparameterisation trick helps us train powerful models even when randomness is involved in generating data.

Let's now look at the decoder, also called the generative network, in a Variational Autoencoder.

The decoder's job is to take the compressed data from the latent space and reconstruct the original input.

It starts with a latent variable — that's a sample drawn from the probability distribution learned by the encoder.

From there, the decoder maps this variable back to the original data space. This process is called reconstruction.

We often write the reconstructed input as: $\hat{x}$. That's just a notation to show it's our best estimate of the original input.

The decoder is usually a neural network with a structure that mirrors the encoder. So, if the encoder reduces the input to a compressed form, the decoder expands it back out again.

In short, the decoder turns low-dimensional, sampled data back into high-dimensional outputs — like images, text, or sound — that look like the original training data.

This is what makes the decoder a key part of generating realistic and meaningful samples.

To train a Variational Autoencoder, or VAE, we use a special loss function made up of two key parts: reconstruction loss and KL divergence.

First, let's talk about reconstruction loss. This measures how well the VAE can rebuild the original input from the latent representation. Our aim is to make this loss as small as possible. A low reconstruction loss means the VAE is doing a good job of recreating the input, leading to more accurate results.

Now, the second part is KL divergence — short for Kullback–Leibler divergence. This measures how different the VAE's learned distribution is from a standard normal distribution. In simpler terms, it checks how well the model is staying within a clean, predictable space. By minimising KL divergence, we make sure the latent space is smooth and behaves nicely, which helps the model generate realistic, varied outputs.

Together, these two losses strike a balance — one makes sure the output matches the input, while the other keeps the latent space well-organised. That's what allows VAEs to generate new data that's both accurate and meaningful.

The Variational Autoencoder uses a loss function called the Evidence Lower Bound, or E-L-B-O. This function has two parts: reconstruction loss and Kullback-Leibler divergence.

The first part is the reconstruction loss, which measures how well the decoder can recreate the original input from the latent variable. It is written as:

$L_{\text{recon}} = -\mathbb{E}_z [\log p(x|z)]$



This means the model tries to maximise the likelihood of reconstructing x from z — or minimise the reconstruction error.

Mathematically, it is written as:

$L_{\text{recon}} = -\mathbb{E}_{q(z|x)} [\log p(x|z)]$

The second part is the K-L divergence, which compares two distributions: the one learned by the encoder, and a standard normal distribution.

It is written as:

$L_{\text{K-L}} = D_{\text{KL}}(q(z|x) \parallel p(z))$

This encourages the latent variables to follow a standard normal distribution, helping the latent space stay smooth and continuous.

Finally, the total loss is the sum of both:

$L_{\text{V-A-E}} = L_{\text{recon}} + L_{\text{K-L}}$

In other words:

$L_{\text{V-A-E}} = \text{reconstruction loss} + \text{K-L divergence}$

Minimising this total loss helps the model learn meaningful representations while still producing accurate reconstructions.

Let's walk through the training process of a Variational Autoencoder, or VAE.

First, the input data is passed through the encoder. The encoder compresses the data into a lower-dimensional representation called the latent space. This helps reduce complexity while keeping key features of the data.

Next, the encoder produces two values for the latent distribution: the mean and the variance. These parameters define a probabilistic distribution, rather than a fixed value, to represent uncertainty in the data.

From this distribution, we draw a sample using the reparameterisation trick. This trick makes the sampling step differentiable, so the model can be trained using backpropagation.

The decoder then takes this sampled point from the latent space and reconstructs the original input data. It aims to produce a version of the input that's as close as possible to the original.

Finally, we calculate the loss, which combines two parts. The reconstruction loss checks how closely the output matches the input. The Kullback–Leibler divergence measures how far the learned latent distribution is from a standard normal distribution.

Together, these steps help the VAE learn a meaningful latent space and generate accurate, high-quality reconstructions.

Let's walk through the reparameterisation trick, a key technique that makes it possible to train Variational Autoencoders efficiently.

First, let's understand the challenge. In VAEs, we sample latent vectors from a probability distribution. But this sampling step is not differentiable. That's a problem, because training neural networks depends on backpropagation, which needs gradients to flow backward through the model. So, if we can't differentiate through the sampling step, we can't train



the model effectively.

This is where the reparameterisation trick comes in.

Instead of sampling directly from a distribution, we re-express the sampling as a deterministic function. This allows us to compute gradients and backpropagate through the sampling process.

The formula for the trick is:

$z = \mu + \sigma \epsilon$

Let me break that down:

z is the sampled latent vector.

μ is the mean predicted by the encoder.

σ is the standard deviation, derived from the log variance.

ϵ is random noise sampled from a standard normal distribution, which is written as $N(0, 1)$.

By rewriting the sampling this way, the model treats it like any other differentiable function.

This also enables gradient descent. That means the model can now update its parameters using the gradient of the loss function—just like in standard neural network training.

So, to summarise, this trick solves the non-differentiability problem by turning stochastic sampling into a differentiable process. That's what makes it possible to train VAEs end-to-end using standard optimisation techniques—even though they rely on randomness.

The latent space in a Variational Autoencoder, or VAE, is a lower-dimensional encoding of the input data. It compresses high-dimensional information into a compact format that still preserves the most important patterns and relationships.

This space captures the underlying variations between different data points. For example, in a dataset of handwritten digits, similar-looking numbers are placed close together in the latent space.

A key feature of this space is smooth interpolation. That means small changes in the latent space lead to smooth, gradual transitions in the output. If you move slightly between two points in this space, the reconstructed images will also change gradually and logically — not abruptly.

This smoothness is crucial for generating high-quality outputs. It helps ensure that the decoder can interpret neighbouring points as belonging to similar data.

Additionally, the VAE can generate new data points by sampling from this latent space and decoding the samples back into the original format. That means it doesn't just reconstruct what it has already seen, but can also produce new, realistic examples.

In short, the latent space allows the VAE to simplify, interpolate, and generate data effectively — all within a well-organised and meaningful representation of the input space.

Variational Autoencoders, or VAEs, are incredibly versatile and have many real-world applications.

First, they can generate entirely new images that look realistic. By learning the underlying patterns in the data, VAEs can create visuals that resemble the original dataset — like generating lifelike faces or nature scenes that never actually existed.



Next, they're useful for filling in missing data. If some values are missing from a dataset, the VAE can estimate those gaps by using what it has learned about the overall data distribution. This helps in completing incomplete records in fields like healthcare, surveys, or sensor data.

They can also detect outliers. By learning what typical data looks like, VAEs can flag anything that strays too far from the usual pattern — which is especially useful for identifying fraud or errors.

Another application is noise removal. VAEs can clean up noisy inputs by reconstructing only the essential underlying signal. This improves the quality of the data for future use, such as training other models or doing analysis.

Together, these applications show how VAEs can be powerful tools for both generating new data and improving the quality of existing datasets.

To build a Variational Autoencoder in Python, we follow four main steps.

First, we choose the right libraries. Most people use deep learning frameworks like TensorFlow or PyTorch to define the neural networks. For data operations, NumPy is used to handle arrays and perform efficient mathematical computations.

Next, we define the encoder and decoder. These are the two core neural networks of the VAE. The encoder compresses the input data into a lower-dimensional representation, and the decoder reconstructs the original data from that compressed form.

Then comes the loss function. We combine two parts: the reconstruction loss, which measures how close the output is to the input, and the Kullback–Leibler divergence, which makes sure the learned latent space follows a smooth, continuous distribution.

Finally, we train the model. We use gradient descent to update the parameters and minimize the total loss. This training process helps the VAE learn how to represent and generate data effectively.

Following these steps ensures a structured and repeatable approach for implementing VAEs in Python.

Can you apply what you've learned to build and train a VAE from scratch — and test how well it captures patterns in your own dataset?

Video 4 - Variational Autoencoder (VAE) Implementation

Imagine a model that doesn't just learn from data — it imagines new possibilities. That's the magic of a Variational Autoencoder. By building one in Python, you'll discover how probability and deep learning combine to create powerful, generative models. Let's break down how each part makes it happen.

In this example, we're going to train a Variational Autoencoder—V-A-E—on the M-NIST dataset. This dataset contains thousands of handwritten digit images ranging from zero to nine.

The main goal is threefold.

First, we want to encode these images into a lower-dimensional latent space. This space should be probabilistic, meaning it captures the uncertainty and structure within the data.

Second, we aim to reconstruct the original images using this latent representation. The



closer the reconstruction is to the original, the better the model has learned.

And third, we want to generate new images. We do this by sampling from the learned latent space and decoding those samples into entirely new digit images.

To achieve these objectives, we follow a clear set of steps.

We begin by loading the M-NIST dataset.

Next, we define the architecture of the V-A-E model, specifying how the encoder and decoder will work.

Then we train the model using the dataset.

After training, we visualise the reconstruction results to check how accurately the model reproduces the input digits.

Finally, we sample new points from the latent space and use the decoder to generate brand new digit images.

By completing these steps, we not only learn to reconstruct existing data but also generate new samples that resemble the original dataset. This is the core capability of a V-A-E—learning meaningful, compressed representations of complex data.

To prepare the M-NIST dataset for training a Variational Autoencoder, we start by importing the necessary libraries.

We use NumPy for numerical processing and TensorFlow or Keras to build, train, and manage the model.

From Keras, we import layers and models to define the neural network, the binary cross-entropy function for reconstruction loss, and backend utilities to support custom operations.

We also import the M-NIST dataset, which includes sixty thousand training images and ten thousand test images.

Each image is a twenty-eight by twenty-eight grayscale picture of a handwritten digit from zero to nine.

Finally, we import Matplotlib to visualise the model's output and plot the results.

We begin by loading the M-NIST dataset, which contains grayscale images of handwritten digits from zero to nine.

To help the model learn better, we normalize the pixel values. Originally, these values range from zero to two hundred fifty-five.

We convert them into floating-point numbers between zero and one by dividing each pixel value by two hundred fifty-five.

Next, we reshape the data to include a channel dimension.

So instead of having a shape of twenty-eight by twenty-eight, we make it twenty-eight by twenty-eight by one.

This extra dimension is needed so that convolutional layers can process the images correctly.

Finally, we print the shape of the training data to confirm everything is formatted as expected.

Let's begin by building the encoder and latent space.

We start with an input layer that accepts a 28 by 28 grayscale image.

Next, we apply two convolutional layers using Rectified Linear Unit, or ReLU, activation and a stride of two. This downscales the image and extracts useful features.

After flattening the result, we use a dense layer to learn higher-level patterns.

Finally, we create two outputs: `z_mean` and `z_log_var`, which capture the mean and log variance of the latent distribution.

Now we move on to sampling using the reparameterisation trick.

This allows us to insert random sampling into the model without breaking the gradient flow.

We define a function that takes `z_mean` and `z_log_var`



and draws a sample from a normal distribution.

The formula is:

$z = \mu + \sigma \epsilon$ — where μ is the mean, σ is the standard deviation, and ϵ is the random noise.

This is implemented using TensorFlow's backend to sample from a standard normal distribution.

The final sample z is generated and passed to the decoder.

Next, we construct the decoder, which reconstructs the image from the latent vector.

We begin with an input layer that accepts the sampled vector z .

Then, we use a dense layer to upscale it into a seven by seven by sixty-four tensor.

We reshape it and pass it through two transposed convolutional layers, again using ReLU activation and stride two.

Finally, the output layer uses a sigmoid activation to produce the reconstructed image, with pixel values between zero and one.

To complete the model, we connect the encoder and decoder.

We define the encoder model with three outputs: z underscore mean, z underscore log underscore var, and the sampled z .

The decoder takes z as input and generates the final image.

Finally, we create the full Variational Autoencoder, or V-A-E, by chaining the encoder and decoder together.

In this step, we define the loss function used to train the Variational Autoencoder.

The total loss combines two parts: reconstruction loss and KL divergence loss.

First, the reconstruction loss checks how well the model recreates the input image.

We use binary cross-entropy between the flattened input and the output.

The formula multiplies this by 28 times 28 to account for all pixels in a 28-by-28 image.

Next, we calculate the KL divergence loss using the mean and log variance of the latent space.

This term pushes the learned distribution closer to a standard normal distribution.

The formula used is:

negative zero-point-five times the mean of one plus z log variance minus the square of z mean, minus the exponential of z log variance.

We add these two losses together and take the mean across the batch.

This combined loss ensures two things—accurate reconstruction and a well-structured latent space.

It helps the VAE learn meaningful image representations while staying mathematically stable.

Now that we've built the Variational Autoencoder and defined its loss function, let's move on to compiling and training the model.

We use an optimizer called Adam to reduce the total loss, which includes both reconstruction loss and KL divergence. This helps the model learn to rebuild images accurately while also learning useful patterns in the data.

To compile the model, we write:

```
"v-a-e dot compile, optimizer equals Adam, loss equals lambda, x comma y colon v-a-e underscore loss, x comma y comma star encoder of x, zero colon two."
```

Then we train the model using the M-NIST dataset. We run it for 50 epochs, with a batch size of 128.



Once training is complete, we check how well the model performs.

First, we compare the reconstructed images with the original inputs. This tells us how well the model has learned to recreate what it sees.

Next, we sample new points from the latent space and decode them into images. If trained well, the model generates new, realistic-looking digits.

These results show how effectively the VAE has learned the structure of the data—and how creatively it can generate new examples.

During training, it's important to keep an eye on how the loss changes over time. This helps us understand how well the model is learning.

We do this by plotting two lines—one for training loss, and one for validation loss—across multiple epochs.

Training loss shows how well the model is fitting the training data. Validation loss shows how it performs on data it hasn't seen before.

By comparing the two curves, we can check if the model is overfitting, underfitting, or learning well. Ideally, both losses should go down and meet at a low point—this tells us the model is generalising properly.

To visualise how well the Variational Autoencoder performs, we start by selecting random test images from the dataset.

Then we use the encoder to convert those images into their latent representations—this is the compressed format that captures the essential features.

After that, we pass the latent vectors through the decoder to generate reconstructed images.

Finally, we arrange the original, latent, and reconstructed views into a three-row plot—so students can compare the input, the encoding, and the output side by side.

In the first row of our plot, we display the original test images using grayscale.

The second row plots each image's location in the latent space using red dots. This shows where the encoder has placed each image in the compressed space.

We also set axis limits to keep the latent space plots clear and consistent, helping students see the distribution pattern more easily.

In the final row, we show the reconstructed images—these are the outputs from the decoder, reshaped back into 28-by-28 pixel grids.

We then apply a tight layout to make sure everything fits well visually.

Finally, we call the complete function to render the full set of visualisations—original images, latent space positions, and reconstructed outputs.

This side-by-side comparison helps students understand how well the VAE captures the data's structure and generates realistic outputs.

Ready to test what the V-A-E has really learned? Try feeding it new data and evaluate how well it reconstructs unseen patterns. How would you improve the encoder or decoder architecture to boost performance?

Video 5 - Generative Adversarial Networks (GANs)

How do you think a machine learns to create something entirely new — like a photo it's never seen?



Generative Adversarial Networks, or GANs, are a type of machine learning framework introduced by Ian Goodfellow and his team in twenty fourteen.

They're made up of two separate neural networks—one called the Generator, and the other called the Discriminator.

The Generator tries to create fake data that looks real, like artificial images or samples.

The Discriminator's job is to spot whether each sample is real or generated.

These two networks compete in a game-like setup—one trying to fool the other.

Over time, this back-and-forth helps both networks improve.

As a result, GANs can learn to create highly realistic outputs—especially useful in fields like image generation, data augmentation, and creative content development.

Let's break down how GANs actually work.

First, we have the Generator. It takes in random noise—like a string of numbers—and tries to turn it into something that looks real, like a handwritten digit or a photo.

Next comes the Discriminator. It takes in actual data and the Generator's output. Its job is to tell which one is real and which one is fake.

Then comes the training objective. The Generator is constantly trying to fool the Discriminator by producing better and better fake data. And the Discriminator is learning to get better at spotting what's fake.

This push-and-pull setup creates a loop. As both networks improve through competition, the Generator eventually learns to create incredibly realistic outputs.

Let's look at the architecture of a Generative Adversarial Network, or GAN.

It has three main parts.

First, the Generator—often called G. It takes in random noise and tries to generate synthetic data that looks like real data. Think of it as an artist trying to paint a realistic image from imagination.

Next, we have the Discriminator—or D. It's like a judge. It looks at the data and decides whether it's real or fake. It's a binary classifier, which means it gives a yes or no answer—real or generated.

Finally, there's the Game. The Generator and Discriminator compete in what's called a minimax game. The Generator keeps improving to fool the Discriminator, while the Discriminator keeps improving to catch the fakes.

This back-and-forth training helps both models get better over time.

The Generator, or G, is one of the two main parts of a Generative Adversarial Network, or GAN. The other part is called the Discriminator, represented by D. The Generator's main job is to create fake data that looks just like real data.

To do this, the Generator starts with a random noise vector, often called z . Think of z as a string of random numbers — like static on a TV screen — pulled from a known distribution such as Gaussian (also called normal) or Uniform.



Using this random input, the Generator creates an output — for example, an image, a sentence, or a short piece of music. It keeps learning how to improve its output until the Discriminator can't tell if the result is real or fake.

Let's break down the formula:

G of z is the generated output.

The goal is for G of z to be indistinguishable from real data, which comes from a real-world data distribution. This is shown mathematically as:

$G(z)$ is distributed like $p\text{-data-of-}x$.

That means the fake data from the Generator should match the pattern of the real data as closely as possible.

In short, the Generator learns how to turn random noise into convincing fake data — like generating a photo of a face that looks real, even though that person doesn't exist.

The Discriminator, also called D , is one of the two networks that make up a Generative Adversarial Network, or GAN. The Generator, or G creates synthetic data. The Discriminator's job is to judge that data.

It acts like a smart critic. Its main role is to tell whether a data sample is real, from the training dataset, or fake, generated by the Generator.

Let's break down how it works.

First, the Discriminator takes a data sample as input. This sample might be real — for example, a photo from the training set — or it could be fake, created by the Generator.

Then, the Discriminator produces an output score, which is a probability. This score shows how confident the Discriminator is that the sample is real. If the score is close to 1, it believes the data is real. If it's close to 0, it thinks the data is fake.

You can think of the Discriminator as a binary classifier. It's constantly evaluating samples and giving feedback to the Generator. This feedback helps the Generator improve over time and produce more realistic outputs.

Given an input sample, let's call it x , the Discriminator produces an output written as D of x . This represents the probability that x is real.

In simple terms, $D(x)$ tells us how real the Discriminator thinks a given sample is. This process helps train the Generator to get better at fooling the Discriminator by generating more convincing fake data.

In Generative Adversarial Networks, or GANs, loss functions are critical for training both the Generator and the Discriminator. The main goal is to help both networks improve over time. The Generator learns to produce data that looks more and more realistic, while the Discriminator learns to tell the difference between real data and fake data.

Let's look at the Discriminator Loss, written as L_D . This measures how well the Discriminator can correctly classify data. It should label real data as real, and fake data as fake. In this case, real is represented by the number one, and fake by zero.

To succeed, the Discriminator needs to be confident in both directions. It wants D of x — the probability that a real sample is real — to be as close to one as possible. And it wants D of G of z — the probability that a fake, generated sample is real — to be as close to zero as possible.

This leads us to the mathematical expression for the Discriminator Loss:



L_D equals the negative expected value of $\log D$ of x , minus the expected value of $\log$ of one minus D of G of z .

In simpler terms, it measures how well the Discriminator assigns high confidence to real data and low confidence to generated data.

Here, D of x is the probability that the input x is a real sample, and D of G of z is the probability that a generated sample is real. The Discriminator is trying to maximise D of x and minimise D of G of z . This push and pull is what drives adversarial learning in GANs — with the Generator trying to fool the Discriminator, and the Discriminator constantly learning to stay one step ahead.

Now let's turn our attention to the Generator Loss. The Generator's main goal is to create data that looks so real, it can fool the Discriminator into thinking the fake data is real.

This is a key part of how GANs work. While the Discriminator tries to spot fake data, the Generator learns to produce increasingly convincing samples — ones the Discriminator mistakes for real.

The Generator Loss is shown on screen with the symbol L_G . The formula reads:
 L_G equals the negative expected value of the logarithm of D of G of z .

Here's what that means in simple terms:

The Generator starts with a random noise vector — we call that z . It transforms this into a data sample, which we call G of z . The Discriminator then looks at that generated sample and outputs a score — that's D of G of z . This score represents the probability that the sample looks real.

The Generator wants D of G of z to be close to 1 — because that means the Discriminator believes the sample is real. So, in training, the Generator is trying to maximise D of G of z . The higher this score, the better the Generator has done its job.

This leads us to the idea of the Adversarial Game. The Discriminator is working hard to correctly detect fake data. The Generator, meanwhile, is trying to outsmart it — by creating data that passes as real. The Generator is essentially trying to minimise the Discriminator's ability to tell the difference.

This constant push and pull — between the Generator and the Discriminator — is what drives the learning process in GANs. As each network improves, the other is challenged to get better too. That's what makes adversarial training so powerful.

Now, let's bring everything together by looking at the GAN Objective Function. This is the big picture — the function that defines the entire learning process in a Generative Adversarial Network.

In simple terms, the Generator is trying to create data so realistic that the Discriminator cannot tell it's fake.

At the same time, the Discriminator is trying to become a better and better critic — one that can correctly spot fake data while still recognising real data as real.

So how does this show up in the objective?

The Generator's objective is to minimise the probability that the Discriminator classifies the generated data as fake.

It wants the Discriminator to be *confused* — to misclassify fake data as real.

The Discriminator's objective is the opposite.

It wants to maximise its ability to classify real data correctly — and, at the same time,



correctly identify the Generator's fakes.

This sets up an ongoing, adversarial game.

The Generator is always improving its deception. The Discriminator is always improving its detection.

And as they challenge each other, both networks get better.

That's the essence of the GAN Objective Function — a constant push and pull that drives learning forward.

Now, let's walk through the GAN Training Process — a fascinating loop where two networks learn and improve by challenging each other.

We start with the Generator. It begins by creating fake data from random noise. At this point, it has no knowledge of what real data actually looks like — it's just guessing.

Then comes the Discriminator. It's shown both the real data from the training dataset and the fake data generated by the Generator.

Its job? Figure out which samples are real and which are fake.

Once the Discriminator makes its decision, it gives feedback to the Generator.

That feedback helps the Generator understand where it went wrong — and how to do better next time.

Now the process repeats.

With each new cycle, the Generator becomes better at producing convincing data.

At the same time, the Discriminator becomes sharper at spotting the fakes.

This continuous back-and-forth — where one network learns to deceive and the other learns to detect — is what drives the training in GANs.

And over time, both networks improve in tandem, pushing the limits of what these systems can create.

While GANs are powerful, training them isn't always smooth sailing. Let's explore three major challenges that often come up.

First, we have Mode Collapse. This happens when the Generator, instead of producing a wide variety of samples, keeps generating very similar outputs — even though it has access to a large input space. It essentially gets stuck repeating the same types of data, ignoring the full diversity it should be capturing.

Next is Non-Convergence. GANs can be tricky to train because both networks — the Generator and the Discriminator — are constantly trying to outsmart each other. Sometimes, this tug-of-war prevents the model from ever settling into a stable state. It just keeps fluctuating without actually learning or improving.

And finally, there's Training Stability. This is all about keeping both networks balanced. If one learns too fast — let's say the Discriminator becomes too good too early — the Generator won't get helpful feedback and its learning can stall. But if the Generator becomes too strong too soon, the Discriminator can't catch up, and it starts giving weak or meaningless feedback.

These challenges are why GAN training is still an active area of research — and why finding the right training techniques is key to unlocking their full potential.

Now that we understand the basics of GANs, let's explore some popular variants that solve specific challenges or enhance what GANs can do.



First, there's Deep Convolutional GAN, or D-C-GAN. This version uses convolutional layers—these are especially powerful for handling images. By adding these layers, D-C-GAN makes the Generator more stable and helps it create sharper, more realistic images.

Next, we have the Wasserstein GAN, or W-GAN. This version was designed to fix two major problems in GANs—mode collapse and unstable training. Instead of using the usual loss function, W-GAN uses something called the *Wasserstein distance*, also known as *Earth Mover's distance*. This creates a smoother and more reliable learning signal, helping the GAN train more effectively.

Then there's the Conditional GAN, or c-GAN. This type of GAN lets us guide the Generator by feeding it extra information—like labels or descriptions. For instance, we can ask it to generate an image of a handwritten number '3' or a cat, simply by providing that label. This makes the output more controlled and relevant to what we want.

Finally, there's Cycle GAN. This version is great for translating images from one type to another—even if we don't have paired training examples. For example, it can turn a summer landscape into a winter scene, or a photo into a painting, by learning the unique features of each type and figuring out how to switch between them.

These variants show just how much GANs have evolved, helping researchers tackle tough problems and unlock new creative possibilities.

Generative Adversarial Networks, or GANs, are not just theoretical—they're being used in a wide range of real-world applications. Let's explore five key ways they're making an impact.

First, image generation. This is one of the most well-known uses of GANs. They can create incredibly realistic images of people, objects, or entire scenes that don't actually exist. For example, think of lifelike portraits of people who were never born, or landscapes that are entirely artificial.

Next is style transfer. GANs can take the style of one image and apply it to another. A common example is transforming a regular photo to look like a painting by Van Gogh or Picasso. You can even apply the colour and texture of one image onto another.

Then we have image super-resolution. GANs can take a blurry, low-resolution image and fill in missing details to make it sharper and clearer. It's like digitally enhancing an old photo to high definition.

Another practical use is data augmentation. In machine learning, we often need lots of training data. But when real data is limited, GANs can generate realistic synthetic data to help fill the gap. This extra data can improve how well other models learn and perform.

Finally, there's text-to-image synthesis—a truly fascinating area. Here, GANs generate images directly from text. For example, you might type "a red apple on a wooden table," and the GAN will create an image that matches that description. This connects language with visual generation in a powerful way.

These applications show just how versatile GANs are—and why they continue to drive innovation across creative, scientific, and technical fields.

As GANs have evolved, researchers have developed advanced techniques to improve image quality, training stability, and overall performance. Let's look at three powerful innovations in this space.

First, we have Progressive GANs. This method doesn't generate a full high-resolution image all at once. Instead, it starts by creating a small, low-resolution image—like a pixelated



thumbnail—and then gradually adds layers to increase the resolution. This step-by-step growth makes training smoother and more stable, and helps both the Generator and Discriminator learn more effectively. The result? Extremely realistic, high-quality images.

Next is Self-Attention GAN, also called SAGAN. This technique adds self-attention, a powerful tool originally used in natural language processing, into the GAN structure. It allows the model to understand relationships between distant parts of an image. For example, in a picture of a dog, it helps connect the head with the tail even if they're far apart in the image. This leads to more coherent, lifelike results—especially useful for complex scenes.

Finally, we have Big-GAN. As the name suggests, this approach scales up GANs to generate much larger and more diverse images. Big-GAN uses bigger neural networks, larger batch sizes, and refined training methods to produce high-resolution images across many categories—from animals to everyday objects—pushing the limits of what GANs can do.

These advanced techniques mark a big leap forward in generative AI, opening the door to more stable training and even more realistic, creative, and useful outputs.

How do you think these advanced GAN techniques could be applied beyond image generation—perhaps in healthcare, art, or science? Take a moment to reflect, and share one real-world application you can imagine

Video 6 - GAN Implementation

What does it take to build a GAN from scratch? You'll learn how to implement both the generator and discriminator using real code.

Welcome to our Generative Adversarial Networks (GAN) Demo. In this demonstration, we'll explore how GANs work through a practical example using the M-NIST dataset, which consists of handwritten digits from 0 to 9. The goal of this demo is to train a Generator that can create images of handwritten digits that are so realistic that the Discriminator cannot tell them apart from real handwritten digits.

The M-NIST dataset contains 60,000 training images and 10,000 test images. Each image is a grayscale 28x28 pixel representation of a handwritten digit, ranging from 0 to 9. The small size and simplicity of the images make it an ideal starting point for training our GAN.

Let's quickly recall the basic structure of a GAN. A GAN consists of two neural networks: the Generator and the Discriminator. These networks are trained in opposition to each other. The Generator generates new images from random noise, aiming to create images that look like real handwritten digits. The Discriminator, on the other hand, tries to distinguish between real images from the M-NIST dataset and the fake images generated by the Generator. The Discriminator's job is to correctly identify whether an image is real or fake.

The Generator is trained to continuously improve its output by learning from the Discriminator's feedback. It strives to produce images that the Discriminator cannot distinguish from real ones. Meanwhile, the Discriminator tries to become better at distinguishing between real and fake images.

This ongoing cycle between the Generator and the Discriminator creates an adversarial game, where one network learns to improve its ability to deceive, and the other learns to improve its ability to detect the deception.



Let's begin building our GAN demo. The first step in any deep learning project is to import the libraries we'll need.

We start by importing PyTorch, our main deep learning framework. Inside PyTorch, we use `torch-dot-n-n`, which stands for neural networks. This module provides everything we need to define our Generator and Discriminator—like layers, activation functions, and loss calculations.

Next, we bring in `torch-dot-optim`, which contains optimization algorithms like Adam and stochastic gradient descent. These optimizers help update our model's weights during training.

To work with image data, we use `torch vision`. From this, we import two tools: `datasets`, which lets us load datasets like M-NIST, and `transforms`, which helps us preprocess those images—converting them into tensors and scaling their pixel values.

We'll also import Data Loader from `torch-dot-utils-dot-data`. This allows us to load data in batches, which is important for training speed and efficiency.

For visualizing the output, we use `matplotlib-dot-pie-plot`, imported as `p-l-t`. This helps us display the images generated by our model so we can monitor how training is going.

And finally, we import `num-pie` as `n-p`, which helps with numerical operations outside of Pie-Torch—like array math or reshaping.

Together, these libraries give us everything we need to build, train, and evaluate our GAN model on the M-NIST dataset.

Continuing with our GAN demo, the next step is data preprocessing. This is a crucial step to ensure that our data is in the right format for the neural networks to learn effectively.

In the code, we start by defining a transform pipeline using `transforms-dot-Compose`. This function allows us to apply multiple transformations to each image in our dataset. The first step is `transforms-dot-To-Tensor`, which converts the images from a regular format like a P-I-L image or Num-Pie array into Pie-Torch tensors. This is necessary because Pie-Torch operates on tensors, which are multi-dimensional arrays.

Next, we apply `transforms-dot-Normalize(zero-point-five comma zero-point-five, zero-point-five comma zero-point-five)`. This step normalizes the pixel values of the images. Normally, the pixel values in an image range from 0 to 255, but to help the model train more efficiently, we scale these values to range from minus-one to one by subtracting the mean (zero-point-five) and dividing by the standard deviation (zero-point-five). This is a common practice in GANs and helps stabilize training.

Once we've defined our transformations, we move on to loading the M-NIST dataset. We use `datasets-dot-M-NIST(...)` to instantiate the dataset and specify the directory where the data should be stored. If the data isn't already there, setting `download-equals-True` will automatically download it for us. We also set `train-equals-True` to indicate that we're using the training split of the dataset.

To make sure our model can access the data efficiently during training, we create a Data Loader. This tool will load the dataset in batches. For example, we pass a `batch-size-equals-64` to ensure that 64 images are processed at once during training. We also set `shuffle-equals-True` to randomly shuffle the data at the start of each training epoch. This helps ensure that the network doesn't memorize the order of the images but rather learns general patterns from the data.

This preprocessing pipeline ensures that our M-NIST images are properly formatted and easily accessible for training the Generator and Discriminator.



Now that we've prepared our data, the next step is to define the Generator model.

In our code, we define a class called Generator, which inherits from `n-n-dot-Module`, the base class in Pie Torch for building neural networks.

We begin by calling `super-init`, which initializes the base class. Then we use `n-n-dot-Sequential` to stack layers that the data will pass through.

The first layer is `n-n-dot-Linear` latent dim comma 256) where latent dim is the size of the input noise vector, and 256 is the output dimension. This layer takes our random noise and transforms it into a 256-dimensional vector.

Next, we apply `n-n-dot-Rel-U`, an activation function that introduces non-linearity. This allows the network to learn more complex patterns.

After that, we add more `n-n-dot-Linear` layers: from 256 to 512, then from 512 to 1024. Each layer is followed by `n-n-dot-Rel-U()` for further non-linearity.

The final layer is `n-n-dot-Linear`, 1024, 28 times 28. This layer outputs a vector with 784 elements, which corresponds to the pixels in a twenty-eight times twenty-eight M-NIST image. We use `n-n-dot-Tanh()` to squash the output values between minus-one and one, matching the normalization we applied to our real M-NIST images.

In the forward method, we define how the input `z` (our random noise) flows through the network. The `z` vector is passed through the sequential layers we defined. The output is reshaped into a twenty-eight times twenty-eight image.

Finally, outside the class, we set latent dim equals 100 for our random noise vector and create the Generator by calling `Generator(latent dim)`.

With this, the Generator is ready to start creating its first "fake" handwritten digits!

Next, let's take a look at the forward method. This defines how the input `z` (our random noise vector) flows through the network.

The noise vector `z` is passed through the layers we defined earlier. The output, image, is reshaped into a twenty-eight times twenty-eight image format using the `image-dot-view` function.

We also define the size of the random input vector with latent dim equals 100. This means that our Generator will take a 100-dimensional random noise vector as input.

Finally, we create the Generator by calling `Generator-latent-dim`). This model is now ready to start producing its first "fake" handwritten digits!

Now that we've defined our Generator, let's move on to the next critical component of our GAN: the Discriminator model.

As you can see in the code, similar to the Generator, the Discriminator is also defined as a Python class inheriting from `n-n-dot-Module`. This is the base class for all neural networks in Pie-Torch.

Inside the init method, we again use `n-n-dot-Sequential` to define the layers of the network. The input to the Discriminator will be an image, so the first layer is `n-n-dot-Linear-28-times-28-comma-512`. This layer takes the flattened 28 times 28 pixel image (which has 784 features) and transforms it into a 512-dimensional vector.

Following this, we use `n-n-dot-Leaky REL-U 0.2`. The Leaky ReL-U activation function is often preferred in GANs because it allows a small, non-zero gradient when the input is



negative. This helps prevent the issue of "dying ReL-Us" and promotes more stable training. The 0.2 here represents the negative slope coefficient, which controls how much the function allows negative values.

We then add another linear layer: `n-n-dot-Linear 512 comma 256`, followed by another `n-n-dot-Leaky REL-U 0.2` to introduce more non-linearity.

The final layer is `n-n-dot-Linear 256 comma 1`. This layer outputs a single scalar value. This scalar value represents the Discriminator's prediction of whether the input image is real or fake. We pass this output through a `n-n-dot-Sigmoid` activation function. The Sigmoid function squashes the output into a range between 0 and 1. A value close to 1 means the Discriminator thinks the image is real, and a value close to 0 means it thinks the image is fake.

The forward method processes the input image. The first thing it does is flatten the image from its original shape, batch size times channels times height times width, into a single vector, which is suitable for input into the linear layers. This flattened image is then passed through the sequential layers.

Finally, we instantiate our Discriminator model by calling `discriminator equals Discriminator`. With this setup, our Discriminator is now ready to act as the discerning critic in the adversarial training process, constantly evaluating the authenticity of images it receives.

Now that we've defined both our Generator and Discriminator models, let's move on to Step 5 of our GAN demo: setting up the Loss Function and Optimizers. These components are critical to driving the learning process for both networks.

Let's begin with the loss function. We define the loss function for both networks as `n-n-dot-B-C-E-Loss`, which stands for Binary Cross-Entropy Loss. This is a widely used and suitable loss function for binary classification tasks, such as the one our Discriminator is performing—classifying images as either real, 1, or fake, 0.

Next, we define our optimizers. Remember, we have two distinct networks—Generator and Discriminator—and each requires its own optimizer because they have different goals and are trained separately.

For the Generator, we use `optim-dot-Adam generator-dot-parameters, l-r=zero-point-zero-zero-two`. The Adam optimizer is initialized here for the Generator, and we pass in `generator-dot-parameters` to specify which parameters (weights and biases) the optimizer should update. The `l-r` or learning rate is set to `zero-point-zero-zero-two`, a common starting point for GAN training. The learning rate controls the step size of the parameter updates. Adam is a popular optimizer for GANs because it adapts the learning rate during training.

For the Discriminator, we initialize another Adam optimizer: `optim-dot-Adam discriminator-dot-parameters, l-r=zero-point-zero-zero-two`. This time, we pass `discriminator-dot-parameters` and use the same learning rate of `zero-point-zero-zero-two`. This ensures that both networks are trained independently but in a balanced way.

By using separate optimizers for both the Generator and the Discriminator, we gain full control over their learning rates, which is essential for balancing the adversarial training process.

Now that we have our loss function and optimizers set up, we're ready to move on to the next step in the training loop.

Now we're at the heart of our GAN demo: Step 6, Training the GAN. This is where the Generator and Discriminator play their adversarial game, improving together through each



iteration.

Let's break down the training loop. First, we define a helper function called `generate-noise batch-size comma latent-dim`. This function generates random noise vectors based on the batch-size and latent-dim.

Next, we enter the Training Loop. For each epoch, the outer loop runs through batches of real images from our train-loader. We only need the images, so we discard the labels.

Now, we train the Discriminator. First, we zero out the gradients for the Discriminator by using optimizer `D-dot-zero-grad`. This prevents gradients from accumulating across training steps. We then feed the real images into the Discriminator to get real validity, and calculate real-loss using our criterion. After that, we generate fake images using the Generator, pass them to the Discriminator to get fake validity, and calculate fake-loss.

The total loss for the Discriminator is calculated as `d-loss` equals real-loss plus fake-loss. The Discriminator aims to minimize this total loss, meaning it wants to classify real images as real and fake images as fake. We perform backpropagation with `d-loss-dot-backward` and update the Discriminator's weights with optimizer `D-dot-step`.

Next, we train the Generator. We zero out the gradients for the Generator using optimizer `G-dot-zero-grad`. We generate fake images and pass them through the Discriminator to get fake validity. For the Generator's loss, we calculate `q-loss` using criterion against real labels (since the Generator's goal is to trick the Discriminator into thinking fake images are real). We perform backpropagation with `q-loss-dot-backward` and update the Generator's weights with optimizer `G-dot-step`.

Finally, after each epoch, we print the current epoch number and the average Discriminator and Generator losses. This helps us track progress and check if the training is working as expected.

This back-and-forth between the Discriminator getting better at detecting real vs. fake images, and the Generator getting better at fooling it, is what drives the training forward.

After training, we now want to generate new images to see how well the Generator has learned. To do this, we define the function `generate-and-plot`, which takes the trained Generator, the latent-dim, and the number of images to generate `n-samples`, set to 16 by default.

The function starts by generating random noise using the `generate-noise batch-size comma latent-dim` function. These noise vectors are then passed through the Generator to create `gen-images`.

We use `dot-detach` to create a new tensor that doesn't require gradients, as we are no longer training the model. The `dot-c-p-u` function moves the tensor to the C-P-U, and `dot-num-pie` converts the tensor into a Num Pie array, which is required for plotting with Mat-plot-lib.

After that, we rescale the output images from `-1 comma 1` to `0 comma 1` by adding 1 to each image and then dividing by 2.

Next, we move on to the visualisation part. We create a Mat-plot-lib figure with the size set to 4 by 4 using `p-l-t-dot-figure`, `fig size` equals `4 comma 4`. We then loop through each of the `n-samples` generated images.

We use `p-l-t-dot-subplot 4-comma-4-comma-i-plus-1` to arrange the images in a 4 by 4 grid. Each generated image is displayed using `p-l-t-dot-i-m-show gen-images-of-i-dot-reshape 28 comma 28 comma c-map equals 'gray'`, where we reshape the image to 28 by 28 pixels and use a grayscale colour map, as we are working with M-NIST images.



We hide the axis labels for cleaner display using `p-l-t-dot-axis off` and organise the spacing using `p-l-t-dot-tight-layout`. Finally, we call `p-l-t-dot-show` to display the images on the screen.

This entire process generates and displays our newly created images, showing how well the GAN has learned to synthesize realistic handwritten digits!

Now that you've seen how a GAN is built, trained, and tested—what could you create with it? Try tweaking the architecture, adjusting the loss functions, or training it on a new dataset. The best way to learn is by experimenting—so dive in and start generating!

Video 7 - Comparing VAEs and GANs

How do V-A-Es and GANs differ in the way they generate data? Understanding this will help you decide which model suits your task best. Let's explore their key differences.

Let's take a moment to compare two prominent types of generative models: Variational Autoencoders (or V-A-E-s) and Generative Adversarial Networks (or GANs). While both are powerful for generating data, they approach the problem with different architectures and training objectives, leading to distinct characteristics.

To begin, let's look at their architecture.

V-A-E-s employ an Encoder-Decoder architecture. In this model, the encoder maps input data to a latent space, and the decoder reconstructs data from that latent space. This is a more structured and direct approach to generating data.

On the other hand, GANs use a Generator-Discriminator architecture. Here, the Generator creates data samples, and the Discriminator acts as a critic, evaluating whether the generated data is real or fake in an adversarial game. This competition between the two models is what makes GANs unique.

Next, let's discuss their training objectives.

V-A-E-s optimize for both reconstruction loss and KL divergence. The reconstruction loss ensures the output closely matches the input data, while KL divergence regularizes the latent space to make it more structured.

Conversely, GANs operate based on a minimax game, where the Generator tries to fool the Discriminator into classifying fake data as real, while the Discriminator works to correctly distinguish real data from fake. This back-and-forth is what drives the GAN training process.

When it comes to output quality, there's a clear distinction.

V-A-E-s tend to generate blurry outputs, mainly because they focus on pixel-wise reconstruction, which sometimes results in less sharp images.

In contrast, GANs are renowned for generating sharp, high-quality outputs. Their adversarial nature encourages realism, making them excel at producing images that look convincingly real.

Let's now consider the latent space of both models.

The latent space in V-A-E-s is structured and continuous, meaning it's easy to interpolate between points and create meaningful variations in the generated data.

However, GANs have an unstructured and less interpretable latent space, meaning navigating it doesn't always lead to smooth or predictable transitions in the generated outputs.

As for training stability, there's a notable difference.

V-A-E-s are generally more stable compared to GANs because their objective function is more straightforward and less prone to oscillations during training.



On the other hand, GANs face challenges like mode collapse and instability, requiring careful tuning to ensure effective learning.

Finally, let's look at their applications.

V-A-E-s are typically used in anomaly detection, where they excel at identifying unusual patterns by learning to reconstruct typical data. They are also used for data compression thanks to their efficient latent space representation.

Meanwhile, GANs shine in image generation, where they can create novel, high-quality images. They are also widely used in style transfer, where they can transform an image from one style to another.

To summarize, both V-A-E-s and GANs are powerful generative models, but they are suited to different tasks based on their strengths. Understanding these differences helps in selecting the right model for a given application or challenge.

Now that you've seen how V-A-Es and GANs differ, which one would you choose for your next project—and why? Try experimenting with both to see which works better for different tasks like anomaly detection or image generation. The best insights come from hands-on exploration.

Video 8 – Generative AI Part 1- Demo

Welcome to this demo on training a variational autoencoder or VAE using a data set of dog images. In this walkthrough we will explore the problem statement, the data pipeline, the model architecture and finally see how the trained model reconstructs and generates images. The idea is that you get to learn how do you work with some external data sources.

So we if you see here we have a dogs folder and in this we have around 1000 images of different dogs, and the idea is to train our variational autoencoder which can generate dogs images. That's what the idea is. The problem scenario if you see here we are exploring generative ai using variational autoencoder problem statement.

The goal of this project is to build a generative model that can compress and reconstruct images of dogs. Instead of just memorising the data set our model will learn a latent space, a compressed representation of the data. From this latent space we can not only reconstruct the original images but also generate new realistic dog images that the model has never seen before.

So we will come up with our typical framework of VAE. We will have our encoder latent sampling stage and then decoder and then we will minimise the reconstruction error and also the KL divergence which is helping us to approximate the learn distribution to gaussian distribution. And the reason we want to approximate it to gaussian distribution because once we find that the data follows gaussian distribution we can easily model it.

So that's what KL divergence will do. Now the data description we are working with a folder of dog images stored locally on disc. These are raw images image files in common format like jpg or PNG.

Our pre-processing pipeline will take these images decode them, normalise their pixel values between 0 and 1 and resize them to a uniform size of say 128 by 128 pixel or we'll see which one we are choosing but the idea is that each of the image should have a uniform height and width. It should not happen that some images are 1000 by 600 and some of them are 800 by 400. We have to make sure all the images are of the same size then only the model can work.

We then split the data set into training and validation sets so we can both train the model



and evaluate its performance. Expected output outcome is we will visualise original versus reconstructed dog images, generate completely new doglike images by sampling from the latent space, observe how the loss curve evolves over epochs that's what we will be doing. We start by declaring what this script is and then import the libraries that we need.

This script is basically eventually we can convert it to dogs VAE.py which we are not doing in this section but generally that's what we do. We also provide some detail about what this is this demo is all about and generally it's a good practise to put the author also in a sense once you have created the file.

Now here we are importing libraries os globe math random then NumPy TensorFlow from TensorFlow keras we are picking up layers models and then from TensorFlow keras we are also importing backend as k and matplotlib.pyplot as plt these are the libraries we need.

Then we are defining the configuration settings first is the data directory that you have to set wherever whichever folder the data is there you have to provide that folder path image size we are setting it as 128 pixels you can try 64 for faster training on CPU but if you have a GPU support go with 128 batch size is 32 again you can play with that number of epochs 25 smaller number I have taken but ideally we should take 100 200 Latent dimension how many features we want to focus upon to understand the dog images that's an inherent feature now 16 is just a number we have taken but experiment with it based on the result that you are getting higher number means more deeper features you are exploring but it will increase the computation and training time so you can decide validation split we have kept as 0.1 10 percent will go into testing validation and 90 percent data will go to training seed we are setting as 42 we all should get the same kind of result reproducibility we are also defining an output directory where we will store some files generating this directory if it is already there it will skip it so this is what we have done here.

Now our data pipeline so we now define how images are discovered on disc decoded resized normalised and batched into data sets.

So the first function that we are defining here is list image files and this is a generic function we are writing extensions that are allowed are jpg, .jpg, jpeg, PNG, webp, bmp. I don't have all of these. I think we have only jpg files, but this is just to write in a generic way that if there are files in other formats it will pick up them as well. Then files is a and is an empty list and then we are running for e in extensions so for each extensions file dot extend and then globe dot globe os dot path join folder whatever the folder we have provided comma e so it will generate the file names with the extension. And then we are saying here files, and we are saying f for f all the files if os path get size is more than zero so any file which has a zero bytes. Zero byte that will be excluded basically if any file is corrupted and it does not have anything inside it and then we say if not files then we will say image not found and then return sorted files so it will return a sorted list of file path this is simply returning a file paths list of file path.

Then we are saying decode resize normalise so it will read raw bytes from the disc decode into RGB tensor RGB we have coloured images so red green and blue channels would be there in the images so it will read raw bytes from the disc decode into an RGB tensor and it will convert pixel values to float 32 in the rain and also it will ensure the values are between zero to one and it will resize to predefined size that we have decided so here it is simply saying tf.io input output read file from path and then decode image pick the image channels three because we have RGB expand animation is equal to false we don't have any animations image convert image d type to float 32 it will also convert it the convert the pixel values between zero and one automatically and then we are resizing the image between uh resizing image to 128 by 128 so that's your pre-processing function that we have defined over here.

Okay next what we are doing here is next we will we are defining make data set function now this make data set function will shuffle the file path deterministically so that it shuffles the images every time we are loading the data for the training purpose we are loading 32



images at a time that's our batch size and we are shuffling the data so that it does not learn the same images again and again split into train and validation set wraps path into a `tf.data` set maps each path to a decoded image batches and prefetches for performance.

So here we are saying it will take a folder as an input batch size that we have defined 32 validation split 0.1 seed all files is equal to we are applying the list image files from the above and providing the folder path here then random dot random seed shuffle all files so we are shuffling the file so that it's random selection of images finding the total number of images and then validation we are finding how many images will go into the validation data and then we are saying validation file is equal to all files colon number of validation file so up to the number of validation indexes will go to the validation data rest will go into the train data so that's what the function we have defined.

So that's a function we have defined and then here we are defining another function within a function we are saying `ds` from file list files and here we are saying pick up the files as an input and `df` dot data sets it's converting it into a data set file type from tensor slices files so all the slices will so all the fly files will be converted into a TensorFlow data set format then we are saying `ds` dot map lambda and we are defining a function here temporary we are calling a function `p` it takes input as `p` and applies this decode resize norm that we defined over here and then number of parallel calls facing `tf` dot data auto tune means automatically see if parallelisation can happen through GPU if that's possible do it.

And then we are saying `ds` is equal to `ds` dot batch size 32 at a time 32 of them prefetch using GPU so that's the function we define and eventually we are returning `ds` from file list train files that we have defined and then validation files we are also picking up what's the length of train files and validation files so that's what we have done here.

And now we're going to call these functions I'm saying here train `ds` validation `ds` sizes is equal to make data set data directory will provide and it will give me the detail as how many uh how many images are there in train how many in validation that's what we are printing.

And now comes your model architecture so your encoder and decoder so we are defining the encoder over here a stack of strided convolutions down samples the image while extracting features and then flattening to a vector, so it takes the image input if you look at the input shape is 128 by 128 and then three channels that's the shape of the input image.

On top of that we are applying a convolutional layer 32 filters all three by three strides we have given to feel free to experiment with that padding is same so ensuring that the result has the same configuration as the input matrix activation is radio this is applied on top of the previous layer which is inputs then another convolutional layer applied on top of the previous layer which is acts another convolutional layer another convolutional layer we are increasing number of filters going deeper in the images and then eventually we are flattening it out everything after flattening it out we are connecting it to a dense layer which will have 256 nodes activation is radio.

And then this is where we define our z mean and z log variance what we are doing is we are learning the variance at the log level and then here layers dot dense 16 units name would be z mean that's what we want to learn we want to learn μ and σ of every feature that we are trying to understand here 16 features, so we want to learn the distribution of all these 16 latent dimensions.

So here we are providing a latent dimension name is this on top of x so till here we have 256 nodes and then it is getting connected to these two layers each one will have 16 node one for the μ one for the σ and this will all get connected here and it will this will



also get connected over here different weights and then it is learning μ and σ for those 16 dimensions so here that's what we are defining.

So if you see here the mean layer is on top of x even the sigma layer is on top of x so that's what we are doing and x is the last layer is 256 nodes then we are defining a sampling function which takes some input as arguments which is basically the mean and sigma value log variance and mean and then epsilon is some random noise so we are saying $k \cdot \text{dot}$ that is back end dot random normal shape is equal to $k \cdot \text{dot}$ shape z underscore m so that's a sampling we are doing.

And then we are returning it like this μ plus exponential y exponential because my variance is at the log level and if I want to get the variance I need to do e to the power log of variance which will return me variance so I have learned we have learned the variance at the log level to convert it into variance level we are doing exponential and then we are multiplying by 0.5 just to control it multiplied by ϵ now this will generate the random sampled values from the learned distribution which we will use to construct the images.

So now next part is your z value which is layers dot lambda sampled value pick up that based on z mean and z log variance it's on top of it with the sampled values and then encoder is equal to models model dot so we are creating our encoder output entirely encoder model basically we are creating here so models dot model inputs it will take and then z mean z log variance and z these are the things that require this particular model object and name is equal to encoder and then we are printing the encoder summary so encoder is created.

Now the decoder part so the decoder part goes like this latent inputs is equal to layers dot input shape latent dimensions is basically going to pick up 16 values, and from those 16 values it's going to reconstruct the image so we are saying connected to dense layer where image size 128 divided by 16 that's a number of nodes multiplied by image size divided by 120 sorry 16 multiplied 256 this is just a way of writing it if you don't want to use this method you can use some fixed number of nodes but generally people use this method of finding how many nodes should be there but as I mentioned feel free to write any number you want. If you want to simply write 1024 write it no problem then activation is redo on the top of latent inputs reshapes it into image size divided by 16 image size divided by 16 256 values okay and then it's top of d and then we here we have convolutional 2d transpose this which is opposite of your convolutional layer. So convolutional layer that we generally use it is compressing the data 2d will take to the higher dimension. so we are applying filters 128 of them and what we are essentially doing if we have this from here we are trying to project into higher dimension higher pixel values. so we are increasing number of cells number of rows and number of columns so that's what convolutional 2d transpose will do opposite of it because we want to generate the image now.

So 128 then 64 32 strides are same padding is same activation ReLU and eventually it gives us back 3 by 3 convolutional 2d transpose 3 by 3 stripes to padding same activation sigmoid and if you look at the output over here which I'll show you later on last set here it returns us 128 by 128 image see here that's a summary of it.

So eventually we get back 128 by 128 image which I'll show you, so this is what the output we are defining and then the decoder model object takes latent input and outputs it is this is the output it is giving so this is the input, and this is the output and name is decoder dot summary and now let's look at this summary here.

So your encoder takes 128 by 128 image 3 channels then compress it to 64 by 64 matrix with 32 channels then 32 by 32 64 channels 16 by 16 128 channels 8 by 8 256 channels we flatten it out 16 384 values connect to 256 values from there μ and σ 16 and 16 each z lambda is what we are learning and then the decoded part if you see it takes only 16 values and then it is connected to a dense layer and then from there we are reshaping



it to 8 by 8 256 channels.

Then convolutional 2d transpose which will convert this into higher dimension from 8 by 8 matrix it converts into 16 by 16 matrix with 128 channels then 32 by 32 matrix 64 channels 64 by 64 matrix 32 channels which gets converted into 128 by 128 matrix 3 channels which will be equivalent to our original image with 3 channels means red, green and blue so that's what the output would be 3 channels red green and blue 128 by 128 so that's what the idea is.

Now this this we have defined now we'll define a class variational auto encoder which inherits the properties and methods of keras model we initialise this class by defining the encoder so we say self.encoder is equal to encoder so it takes encoder and decode as an input and any other keyword arguments and then metric tracker self.total loss tracker tf.keras metric.mean name is equal to loss then reconstruction loss tracker we are defining tf.keras metric.mean name reconstruction loss and similarly you have KL loss as well.

So when we are initialising this variational auto encoder the above defined encoder and decoder will come as an input and then it will instantiate using those into an object then we define a metric over here that's a property and we say self.total loss tracker return that reconstruction loss and KL loss tracker so return all that.

Then we are defining a call function which takes some inputs which will be your learnt values so z mean z log and z value and self.encoder you apply encoder on the inputs basically you apply inputs are nothing but images and once you apply when you call this function called call it will apply encoder on that and will return the self.decoder z that's what it does so it calculates the z value and then returns decoder z.

Then we are defining compute losses function it will compute the loss so here z mean z log z is equal to self.encoder and then we take x which is your input image here training is equal to training so we are defining here a boolean parameter whether your training is true or false that's what we will be doing and then we are calculating reconstruction is equal to self.decoder on z training is equal to training whatever you have provided the parameter.

Reconstruction loss would be reduced mean is basically finding the mean find the sum of x minus reconstruction so take the image and find the get the reconstructed image find the difference of that and that becomes your error so basically this is mean squared error that we are calculating between the pixel values what was the pixel of the original image and what is the reconstructed pixel value find the difference of that and square it that becomes your reconstruction loss so that's what we are doing over here.

Then KL loss we are saying minus 0.5 times tf reduce mean again reduce sum and then this part here we have done earlier as well in the in the other demo this is the simplified calculation of KL loss where we say 1 minus log of variance minus your mean squared minus your variance so basically your KL loss can be entirely simplified like this mu squared minus variance squared this is what we calculate for KL loss that's what we are doing over here doing it for all the images summing it up and then finding mean of it.

And then the total loss is equal to reconstruction loss plus KL loss and then return the total loss reconstruction loss and KL loss so KL divergence loss so that's what we have done here.

And then now the train step where we take data as an input x is equal to data square bracket 0 if the instance is a tuple hence the data itself and then here with tf dot gradient tape which will keep track of the gradients which we use to update the weights total loss reconstruction loss scale loss is equal to self-dot compute losses on x training is equal to true calculate the gradients so tf dot gradient using total loss it will apply on the self-trainable weights to update the weights basically calculate the gradients and then this is where the weights are getting updated using the gradients that we have calculated.



And then update the total loss reconstruction loss and KL loss based on the updated weights and return the loss reconstruction loss and KL loss that's what we are doing over here in my training step.

And then the test step everything remains same it's just that your training part would be false see here we are not training it, but we are simply calculating everything what is the loss on test data.

And then we define our VAE is equal to VAE which takes encoder and decoder as an input we compile our VAE optimiser is equal to keras optimiser Adam we don't have to define loss here because we have defined custom loss and then we start training the model you say VAE train data set validation data set epochs that we have decided 25 model runs it gets trained.

And then we have defined some utility function to visualise and save and also to plot later so here's show and save reconstructions, so we are saying take the data set n is equal to 8 how many images you want to work with f name file name is reconstructions dot PNG we say batch is equal to next iter data set

So whatever the data set is, pick up the next image in that images is equal to batch colon N eight images is what we are picking up. Then we get the Z value from here and code dot predict images. We'll give you the Z mean and the reconstruction would be based on Z and the square mean. Then we are plotting that, and we are saving that image.

Instead of saving, we can show also in the notebook itself, which I'll, which I think I have done later. I'll show it to you. So this is simply picking up the images, converting into NumPy array. So reconstructed images converted into NumPy and then, sorry, this is the original image. Reconstructed images here and showing both of them and storing it.

That's what we are doing here. And then here we are also defining a function sample, random images. Which is basically picking up random samples. And then we are getting generator. So basically here decoded predict on the Z value, which is a random value. And then columns, rows, how many rows, how many columns we want to plot.

That's what it is basically. And uh, here we are plotting the generated images, you know. In a grid kind of structure. That's what we are doing over here. And then here we are plotting the losses. We have the history, we take the input as history, and we plot it. And we are plotting the validation loss and the training loss.

That's what we are doing over here. And once we run this, this will store the outputs over here. See this reconstruction PNG. So after 25, it epochs reconstruction is this, which is not up to the mark at all because I have run only for 25 EPOCHs. You have to run it for more, and that's what it's doing here.

And these are the generated images. Again, does not make sense, but understand we have run only for 24 so far. We have to increase it. And then just making slight changes over here instead of storing it. Same functions, but it is showing images over here. That's the only difference. So this is what we have done.

We have learned how to pick up images from an external source instead of built-in data set and build model around it. What I would say is that experiment with the number of epochs and other hyper parameters. The first thing you can do is increase the epochs, run it for say, a hundred, 200 epochs and see how the images are coming.

It'll come very nicely. So that was the demo on a sum external data sources. I hope this is fine with all of you and do the experiment and share the results.